

SimuFragDB

Ashok (IMT2023083) Sathvik (IMT2023001)
Sasank (IMT2023120) Kapil (IMT2023052)

February 5, 2026

The complete source code for this project is available at: [GitHub Repository](#)

1 Introduction

This report outlines the implementation and evaluation of SimuFragDB, a sharded PostgreSQL-based system designed to distribute student and course data across multiple database fragments. The system utilizes a custom router to handle horizontal partitioning and ensures data consistency through standardized JDBC connectivity.

2 Data Aggregation Approach

To handle queries in the fragmented database environment, two execution strategies were implemented depending on the type of operation.

For localized operations like `INSERT_STUDENT`, `INSERT_GRADE`, `UPDATE_GRADE`, `DELETE_STUDENT_FROM`, and `READ_PROFILE`, the system uses the `Router` class to map a `student_id` to a specific fragment. This ensures that queries are executed only on the fragment containing the required records, thereby improving efficiency and minimizing inter-fragment communication.

For analytical queries such as `getAvgScoreByDept` and `getAllStudentsWithMostCourses`, the following approach was taken:

- **Fragment Selection:** The client selects one fragment randomly from the available fragments in the `connectionPool`.
- **Local Processing:** The selected fragment performs all required joins (e.g., joining `Grade` and `Course`) and executes aggregation or grouping operations on the data stored within that fragment.
- **Result Formatting:** The Java application retrieves the result from the selected fragment, formats it into the required output string, and returns it to the client.

This approach reduces query overhead and simplifies execution, although aggregation results are computed using only the data present within the selected fragment rather than the entire distributed dataset.

3 Challenges and Resolutions

During development, several technical hurdles were encountered:

1. **Connection Synchronization:** Initially, the system failed if one fragment was offline. I resolved this by implementing a strict check in `setupConnections()` that throws a `RuntimeException` if any database node is unreachable, ensuring system integrity from the start.
2. **Incomplete Aggregation Results:** A significant challenge was ensuring that the `READ_SCORE` query accounted for departments split across multiple shards. I resolved this by moving away from random fragment selection and implementing a full-cluster scan for all analytical read operations.
3. **Environment Management in WSL:** Setting up multiple PostgreSQL instances within a WSL environment required careful user permission management and service startup handling.

4 Evaluation and Results

To verify correctness, the system was tested against a control group (a single database containing the full dataset) and the sharded SimuFragDB implementation.

4.1 Execution Metrics

The execution time was recorded by the driver while processing the `workload.txt` file:

- **Total Execution Time:** [6286] ms

4.2 Accuracy Metrics

Accuracy was calculated by comparing the `output.txt` from the sharded system against the `expected_output.txt` from the single-database control.

- **Total Number of Query Output Lines:** [2642]

The Accuracy Percentage was determined using the following formula:

$$\text{Accuracy Percentage} = \frac{\text{matchinglines}}{\text{totallines}} \times 100$$

Final Accuracy Percentage: [29.488220958570267]%